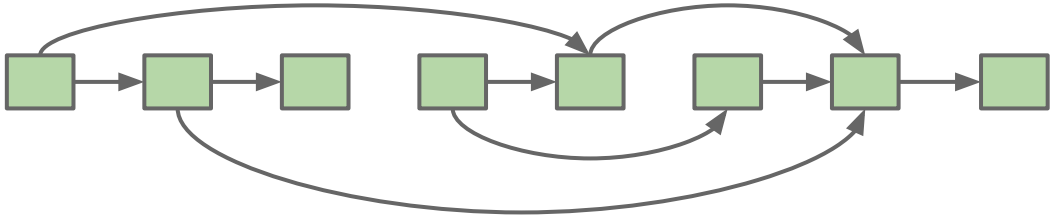


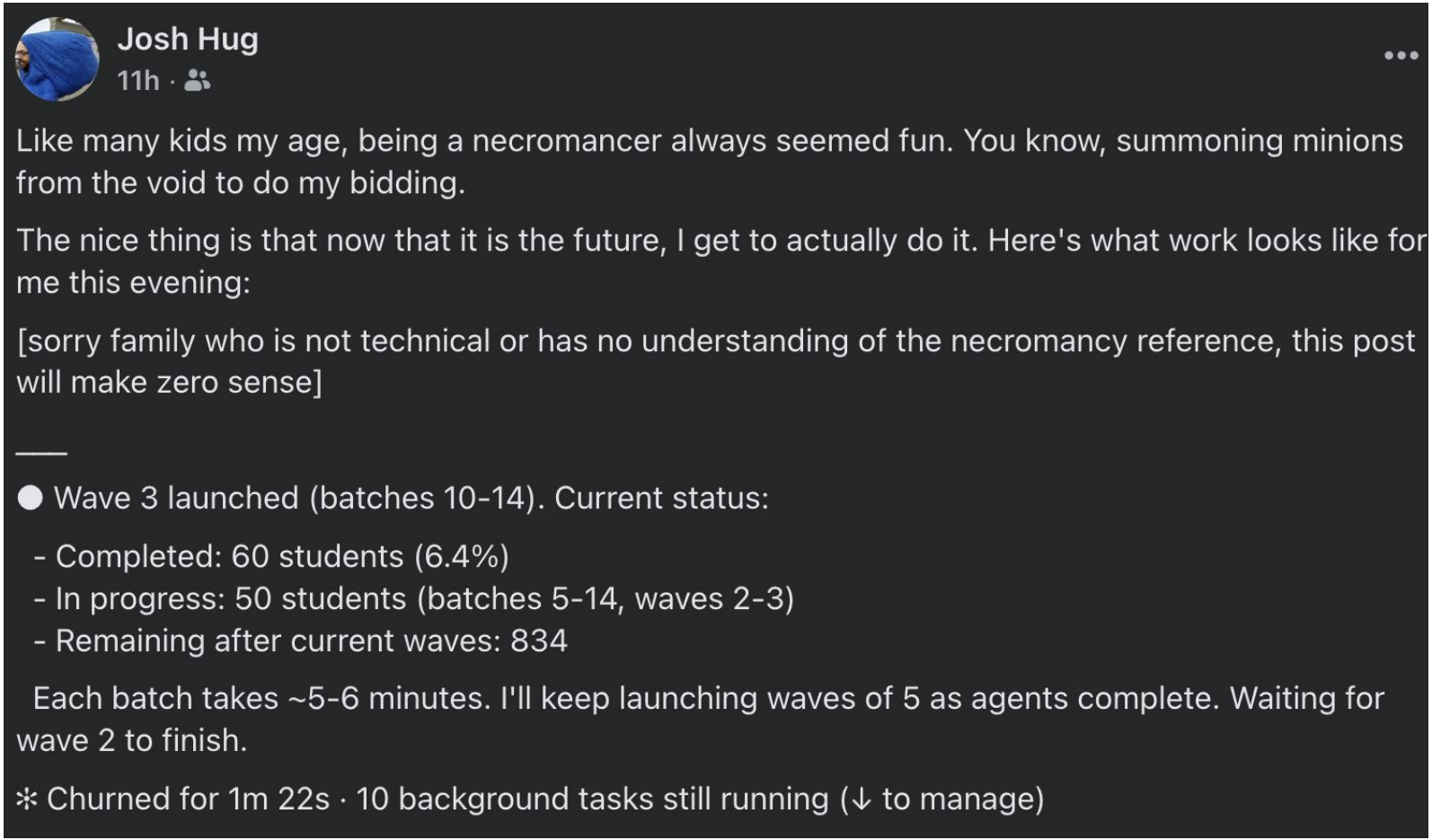
Lecture 25 (Graphs 5)

Directed Acyclic Graphs

CS61B, Spring 2026 @ UC Berkeley

Josh Hug and Kay Ousterhout

Quick Note: Teaching in the Future is Amazingly Fun



Self-report on the Project 4B Process

- That's wild. Here are the numbers:

  Peak: ~13,600 tokens/sec with ~92 concurrent agents running at once (not 20 — the agents from multiple batches overlapped since they were launched in the background).

  Key stats:
  - 92 agents running simultaneously at the true peak (multiple batches of 20 overlapping)
  - ~123M total tokens for the session (7.8M in agents, 115M in the parent orchestrator's growing context)
  - 205 agents completed successfully, 228 hit the rate limit wall and died
  - The main burst was just 25 minutes (13:20–13:46 UTC), where 71% of all review work happened — then rate limiting kicked in and killed everything
  - Average agent: ~38K tokens, ~3.8 min, ~150 tok/s individually

  So at peak, we were burning through tokens at roughly the speed of reading a novel every 5 minutes.

Self-report on the Project 4B Process

● That's wild. Here are the numbers:

Peak: ~13,600 tokens/sec with ~92 concurrent agents running at once (not 20 — the agents from multiple batches overlapped since they were launched in the background).

Key stats:
- 92 agents running simultaneously at the true peak (multiple batches of 20 overlapping)
- ~123M total tokens for the session (7.8M in agents, 115M in the parent orchestrator's growing context)
- 205 agents completed successfully, 228 hit the rate limit wall and died
- The main burst was just 25 minutes (13:20–13:46 UTC), where 71% of all review work happened — then rate limiting kicked in and killed everything
- Average agent: ~38K tokens, ~3.8 min, ~150 tok/s individually

So at peak, we were burning through tokens at roughly the speed of reading a novel every 5 minutes.

● Back-of-envelope with Opus pricing ($15/M input, $75/M output, $1.875/M cached input):

The big unknown is the input/output split and cache hit rate. For this workload (agents mostly reading PDFs + guide, writing ~500-word feedback):

- Agents (7.8M tokens): ~90% input, ~10% output → ~7M × $15/M + ~0.8M × $75/M = ~$165
- Parent orchestrator (115M tokens): Mostly cached context re-reads as the conversation grew. Maybe ~1–2M output, rest is input with heavy caching → ~100M × $1.875/M + ~13M × $15/M + ~2M × $75/M = ~$535

Rough total: ~$700, give or take a few hundred depending on actual cache hit rates.

The parent orchestrator dominates the cost — not because it's doing much work, but because it keeps re-sending the growing conversation context on every turn. The 444 actual review agents were the cheap part.

Goals of Today

Today, to practice our problem solving skills, we'll work through some very challenging A-level problems using the tools we've already learned about.

Will focus on graphs, but the ideas today are more general.

Graph Problems So Far

Problem	Problem Description	Solution	Efficiency
paths	Find a path from s to every reachable vertex.	DepthFirstPaths.java Demo	O(V+E) time Θ(V) space
shortest paths	Find the shortest path from s to every reachable vertex.	BreadthFirstPaths.java Demo	O(V+E) time Θ(V) space
shortest weighted paths	Find the shortest path, considering weights, from s to every reachable vertex.	DijkstrasSP.java Demo	O(E log V) time Θ(V) space
shortest weighted path (extra topic, out of scope)	Find the shortest path, consider weights, from s to some target vertex (extra topic, out of scope)	A*: Same as Dijkstra's but with h(v, goal) added to priority of each vertex. Demo	Time depends on heuristic. Θ(V) space

Graph Problems So Far

Problem	Problem Description	Solution	Efficiency
minimum spanning tree	Find a minimum spanning tree.	LazyPrimMST.java Demo	O(???) time Θ(???) space
minimum spanning tree	Find a minimum spanning tree.	PrimMST.java Demo	O(E log V) time Θ(V) space
minimum spanning tree	Find a minimum spanning tree.	KruskalMST.java Demo	O(E log E) time Θ(E) space

Topological Sorting

Lecture 25, CS61B, Spring 2026

Topological Sorting

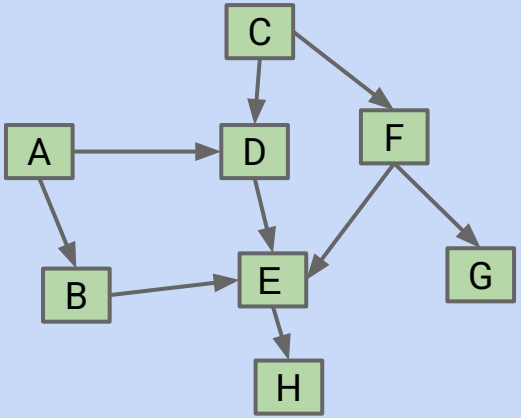
Shortest Paths on DAGs

Longest Paths

Reductions

- Definition
- Reduction to 3SAT (Optional CS170 Preview)

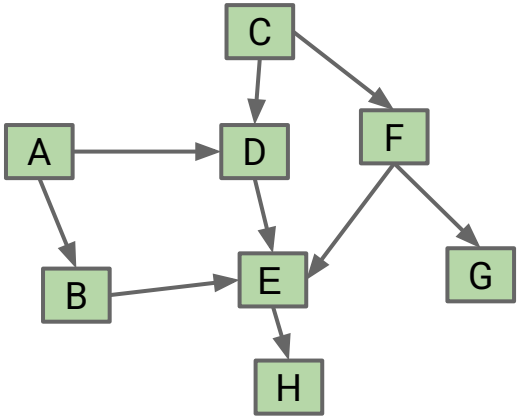
Topological Sort



Suppose we have tasks A through H, where an arrow from v to w indicates that v must happen before w.

- What algorithm do we use to find a valid ordering for these tasks?
- Valid orderings include: [A, C, B, D, F, E, H, G], [C, A, D, F, B, E, G, H], …

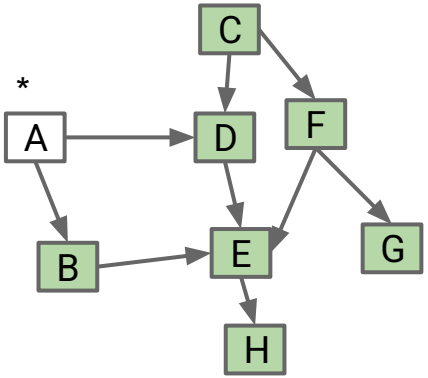
Solution (Spoiler Alert)



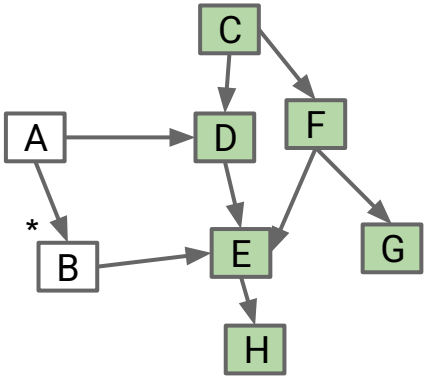
Perform a DFS traversal from every vertex with indegree 0, NOT clearing markings in between traversals.

- Record DFS postorder in a list.
- Topological ordering is given by the reverse of that list (reverse postorder).

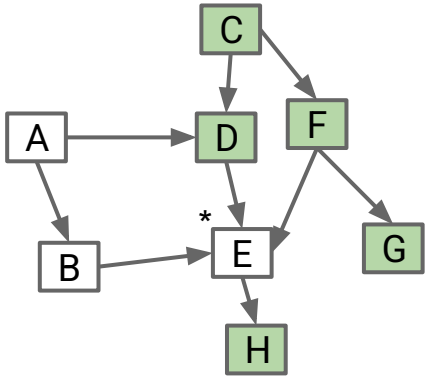
Topological Sort (Demo 1/2)



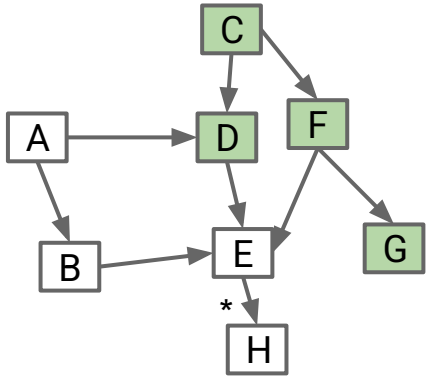
Postorder: []
Call stack: A



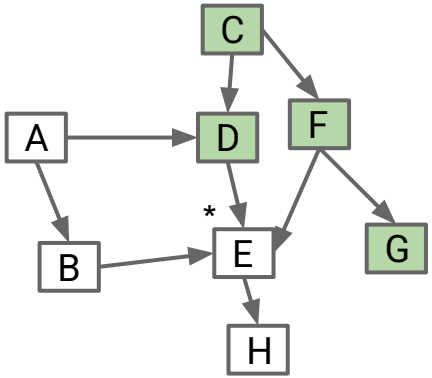
Postorder: []
Call stack: A→B



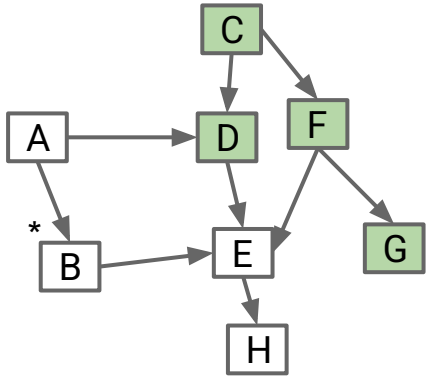
Postorder: []
Call stack: A→B→E



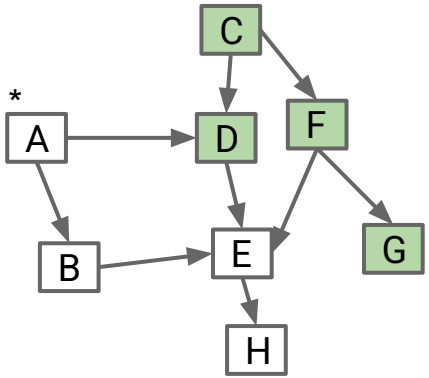
Postorder: [H]
Call stack: A→B→E→H



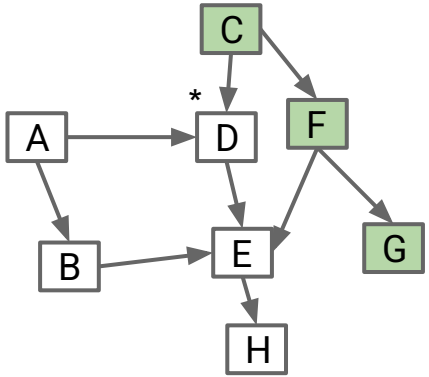
Postorder: [H, E]
Call stack: A→B→E



Postorder: [H, E, B]
Call stack: A→B

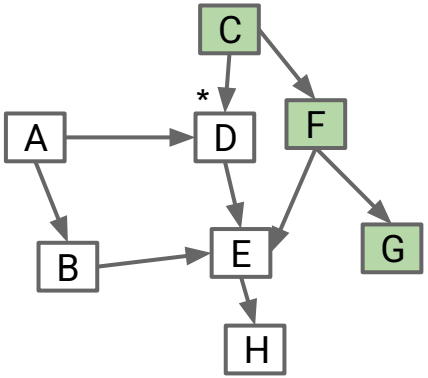


Postorder: [H, E, B]
Call stack: A

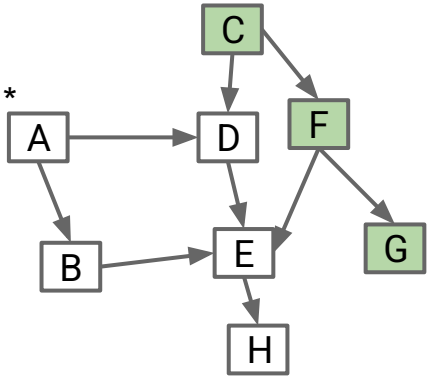


Postorder: [H, E, B, D]
Call stack: A→D

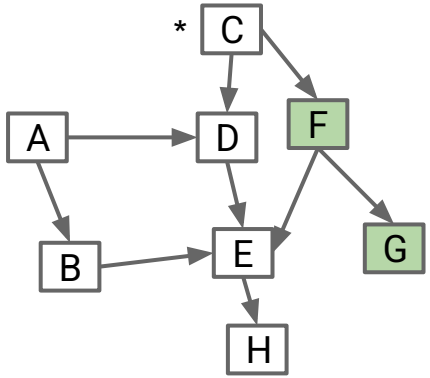
Topological Sort (Demo 2/2)



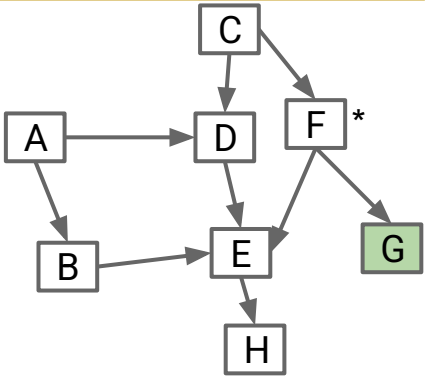
Postorder: [H, E, B, D]
Call stack: A→D



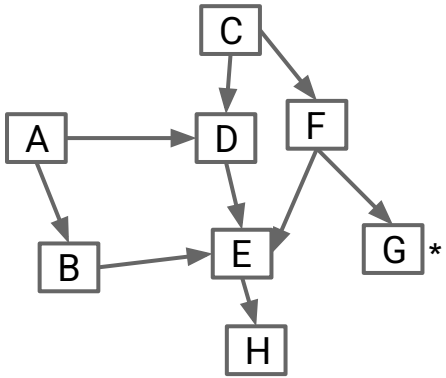
Postorder: [H, E, B, D, A]
Call stack: A



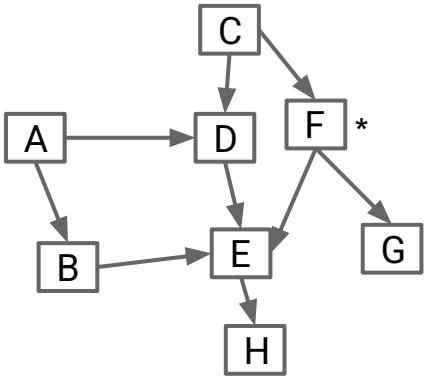
Postorder: [H, E, B, D, A]
Call stack: C



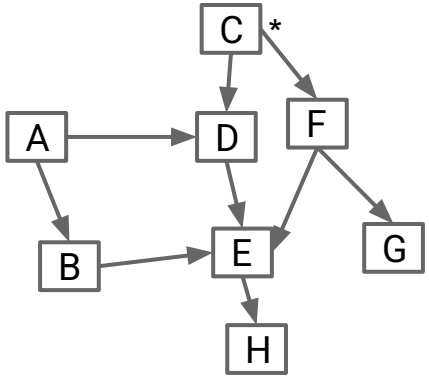
Postorder: [H, E, B, D, A]
Call stack: C→F



Postorder: [H, E, B, D, A, G]
Call stack: C→F→G

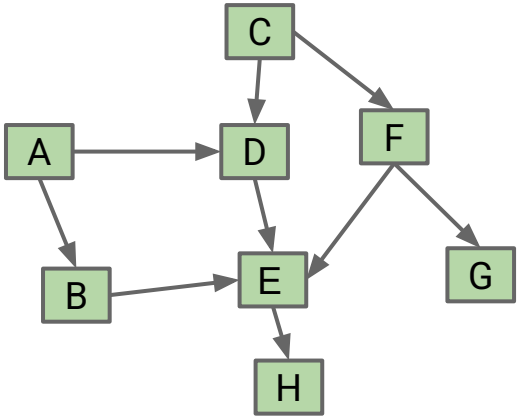


Postorder: [H, E, B, D, A, G, F]
Call stack: C→F



Postorder: [H, E, B, D, A, G, F, C]
Call stack: C

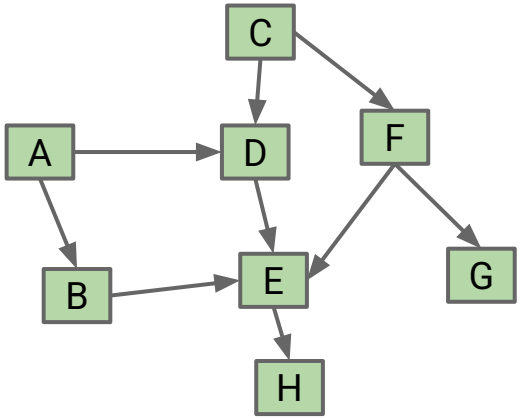
Solution (Spoiler Alert)

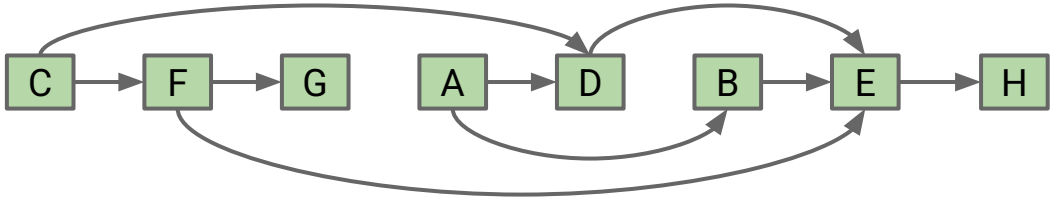


Perform a DFS traversal from every vertex with indegree 0, NOT clearing markings in between traversals.

- Record DFS postorder in a list: [H, E, B, D, A, G, F, C]
- Topological ordering is given by the reverse of that list (reverse postorder):
  - [C, F, G, A, D, B, E, H]

Topological Sort





The reason it's called topological sort: Can think of this process as sorting our nodes so they appear in an order consistent with edges, e.g. [C, F, G, A, D, B, E, H]

- When nodes are sorted in diagram, arrows all point rightwards.

Depth First Search

Be aware, that when people say “Depth First Search”, they sometimes mean with restarts, and they sometimes mean without.

For example, when we did DepthFirstPaths for reachability, we did not restart.

For Topological Sort, we restarted from every vertex with indegree 0.

Question (Extra Slide)

What is the runtime to find all vertices of indegree 0?

Question (Extra Slide)

What is the runtime to find all vertices of indegree 0?

- Interesting thing I did not tell you: You don’t have to.

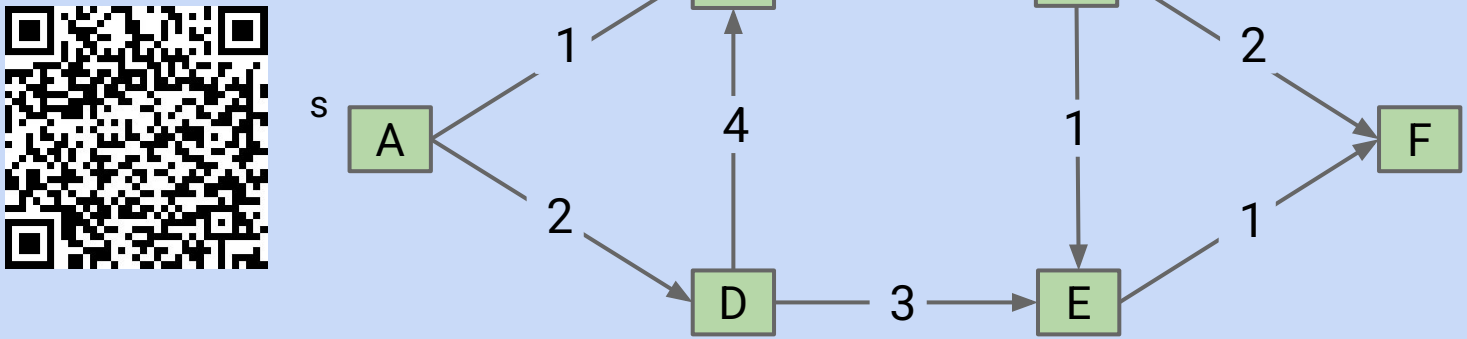
Another better topological sort algorithm:

- Run DFS from an arbitrary vertex.
- If not all marked, pick an unmarked vertex and do it again.
- Repeat until done.

Test Your Understanding: yellkey.com/member

Give a topological ordering for the graph below (a.k.a. topological sort).

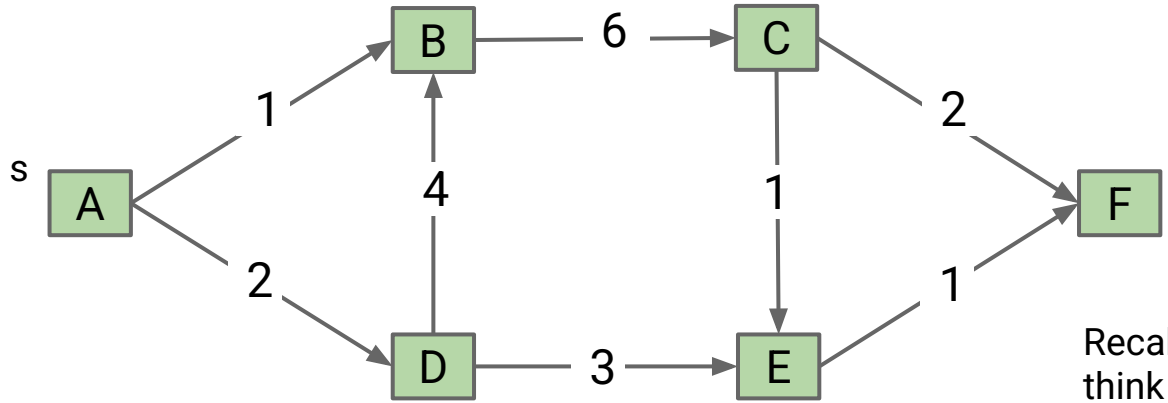
- Give your answer as a comma separated list, e.g. A, B, C, D, E, F



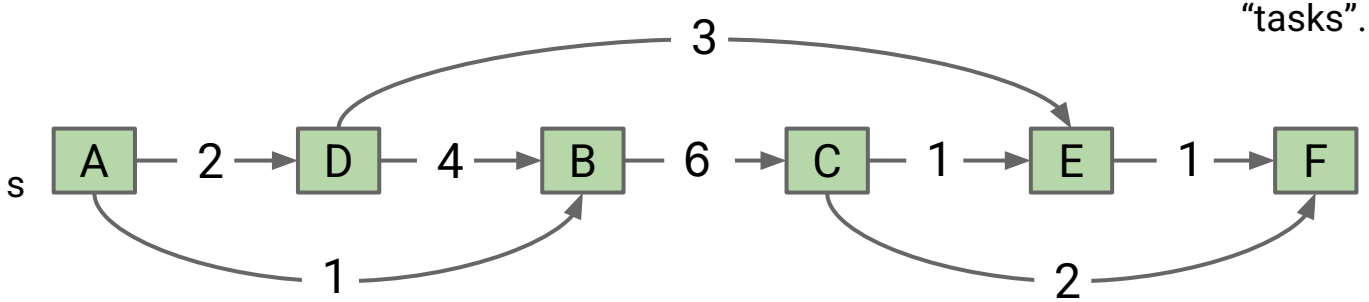
Test Your Understanding

Give a topological ordering for the graph below (a.k.a. topological sort)

- A, D, B, C, E, F (one way to find topo sort: DFS postorder is FECBDA)



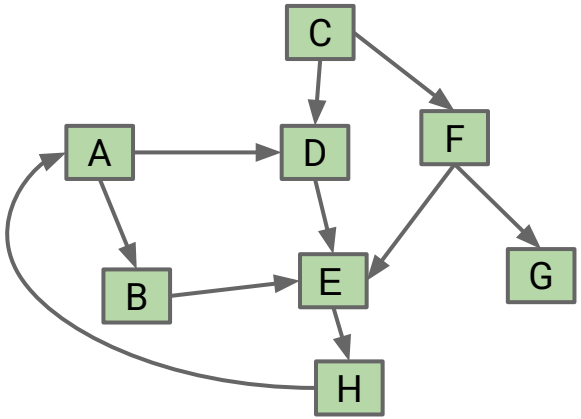
Recall that we can think of topological sort as an ordering of "tasks".



Directed Acyclic Graphs

A topological sort only exists if the graph is a directed acyclic graph (DAG).

- For the graph below, there is NO possible ordering where all arrows are respected.



DAGs appear in many real world applications, and there are many graph algorithms that only work on DAGs.

Graph Problems

Problem	Problem Description	Solution	Efficiency
topological sort	Find an ordering of vertices that respects edges of our DAG.	Demo Topological.java	O(V+E) time Θ(V) space

Shortest Paths on DAGs

Lecture 25, CS61B, Spring 2026

Topological Sorting

Shortest Paths on DAGs

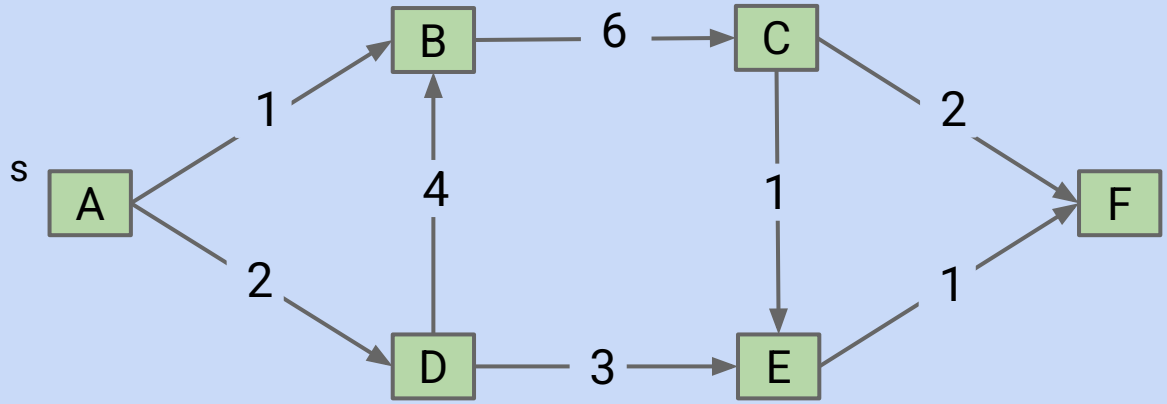
Longest Paths

Reductions

- Definition
- Reduction to 3SAT (Optional CS170 Preview)

Shortest Paths Warmup

What is the shortest paths tree for the graph below, using s as the source?
In what order will Dijkstra's algorithm visit the vertices?

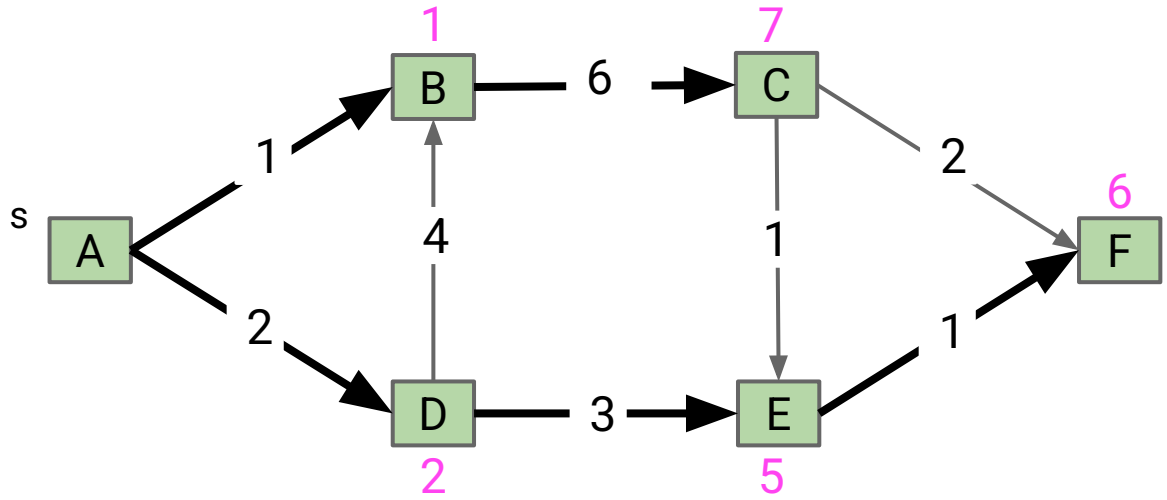


Graph from Algorithms, by Vazirani/Papadimitriou

Shortest Paths Warmup

What is the shortest paths tree for the graph below, using s as the source?
In what order will Dijkstra's algorithm visit the vertices?

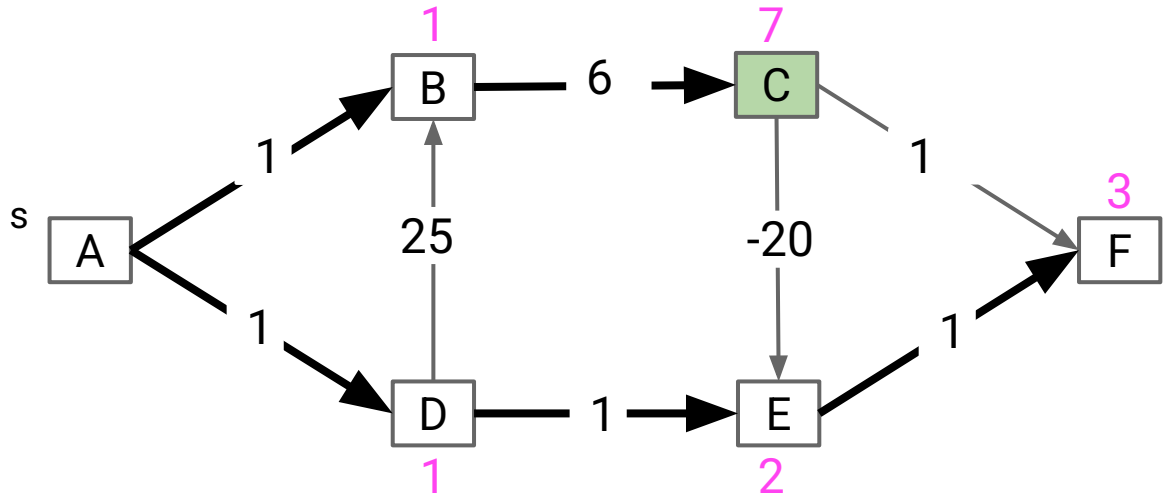
- A, B, D, E, F, C



Shortest Paths Warmup

If we allow negative edges, Dijkstra's algorithm can fail.

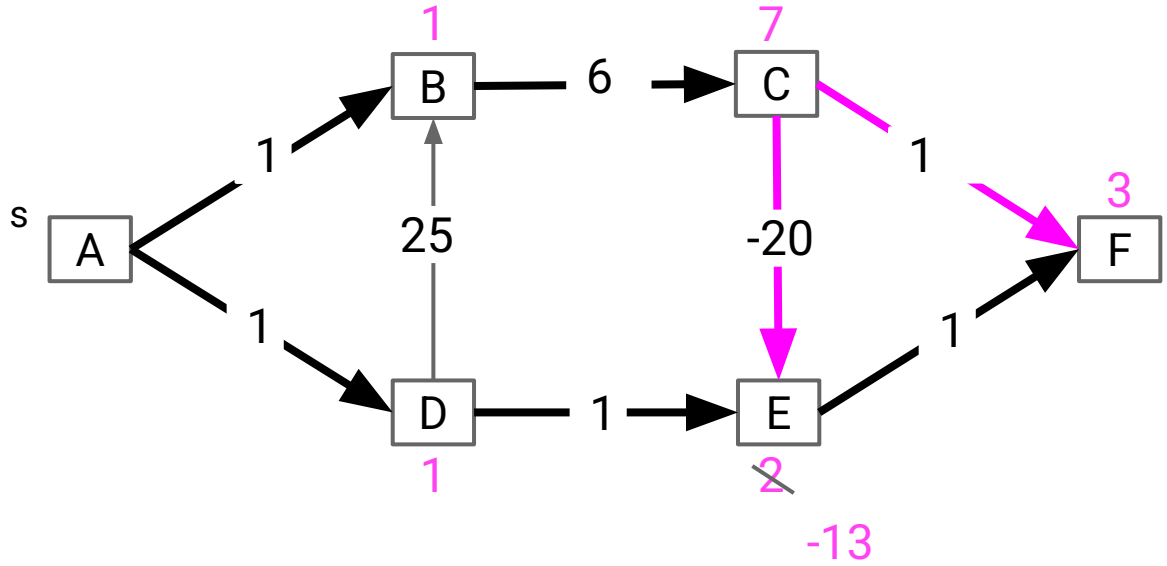
- For example, below we see Dijkstra's just before vertex C is visited.



Shortest Paths Warmup

If we allow negative edges, Dijkstra's algorithm can fail.

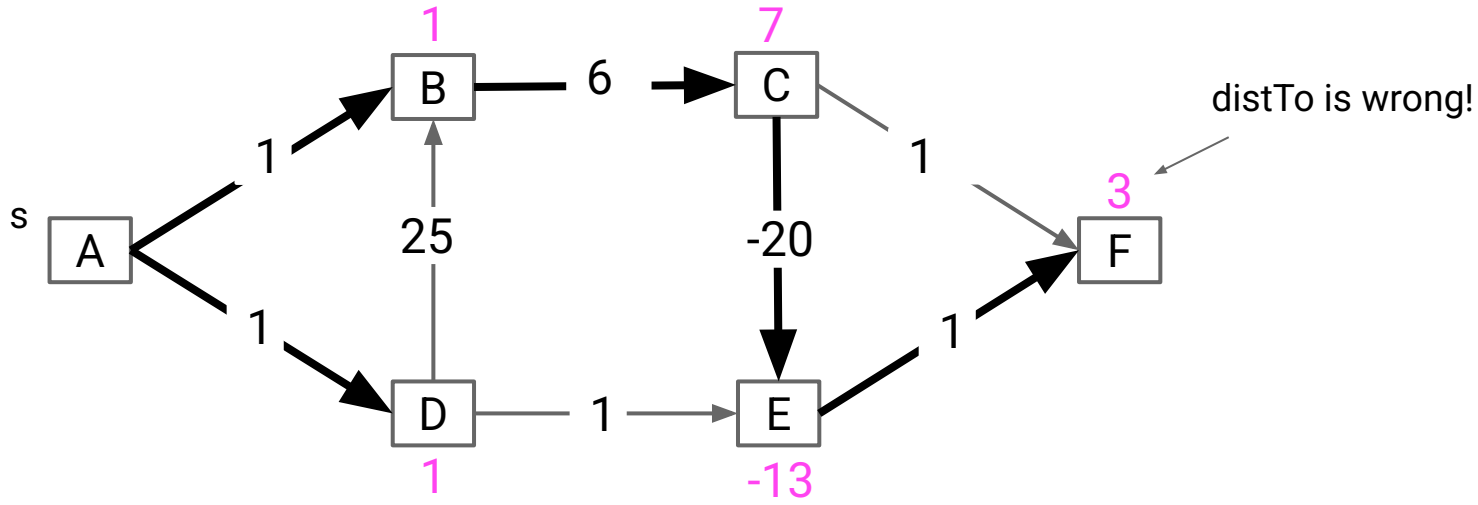
- For example, below we see Dijkstra's just before vertex C is visited.
- Relaxation on E succeeds, but distance to F will never be updated.



Shortest Paths Warmup

If we allow negative edges, Dijkstra's algorithm can fail.

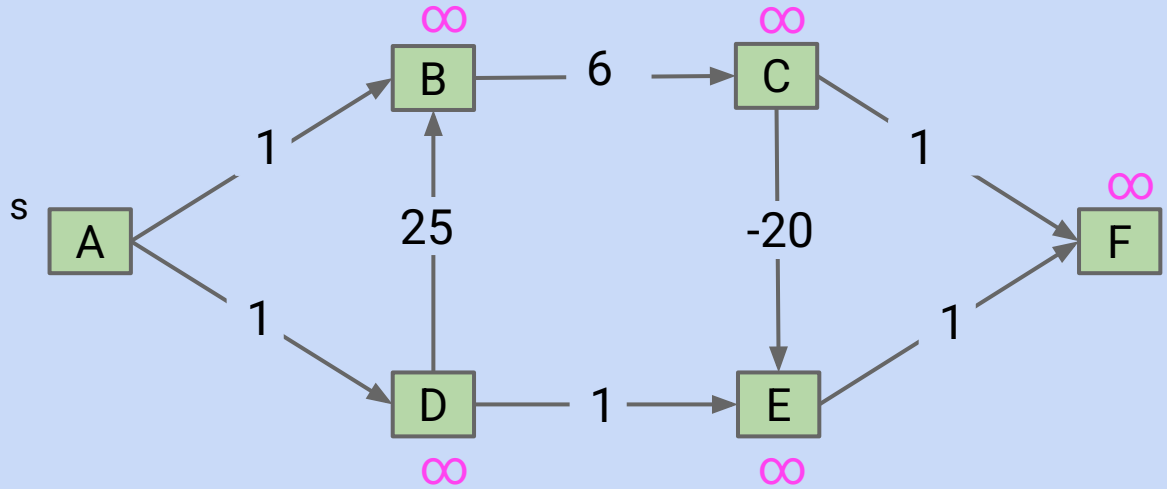
- For example, below we see Dijkstra's just before vertex C is visited.
- Relaxation on E succeeds, but distance to F will never be updated.



Challenge

Try to come up with an algorithm for shortest paths on a DAG that works even if there are negative edges.

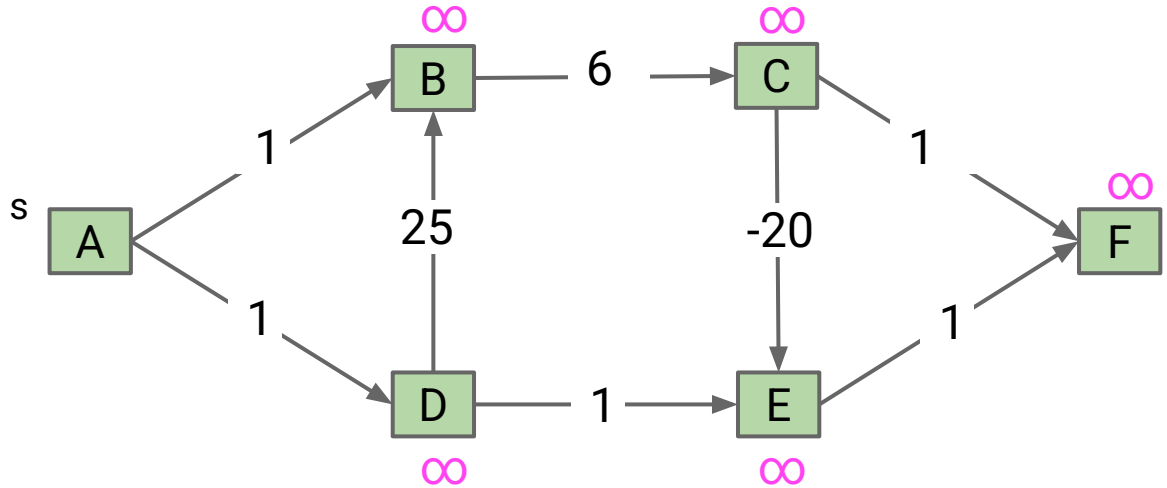
- Hint: You should still use the “relax” operation as a basic building block.



Challenge

Try to come up with an algorithm for shortest paths on a DAG that works even if there are negative edges.

- Hint: You should still use the “relax” operation as a basic building block.

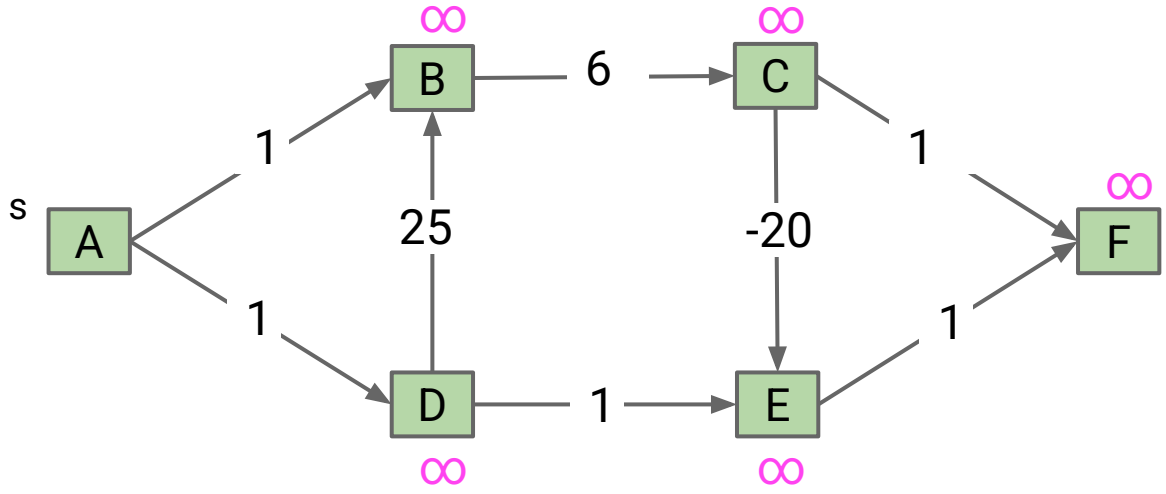


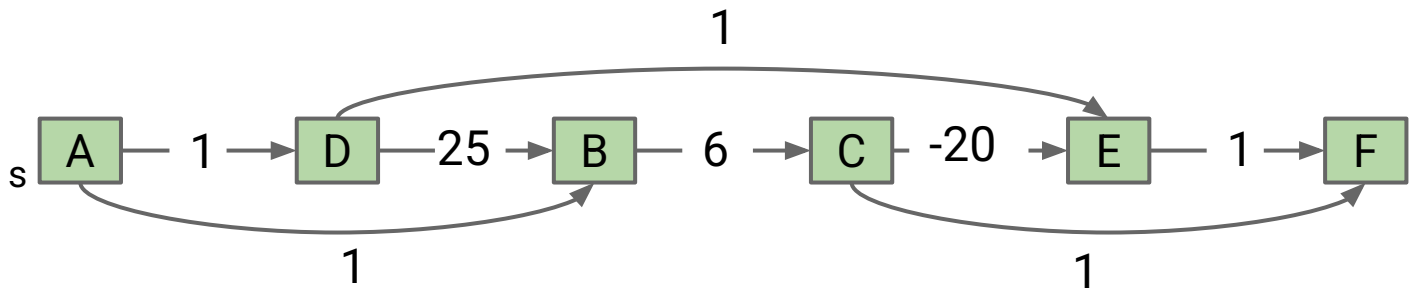
One simple idea: Visit vertices in topological order.

- On each visit, relax all outgoing edges.
- Each vertex is visited only when all possible info about it has been used!

The DAG SPT Algorithm: Relax in Topological Order

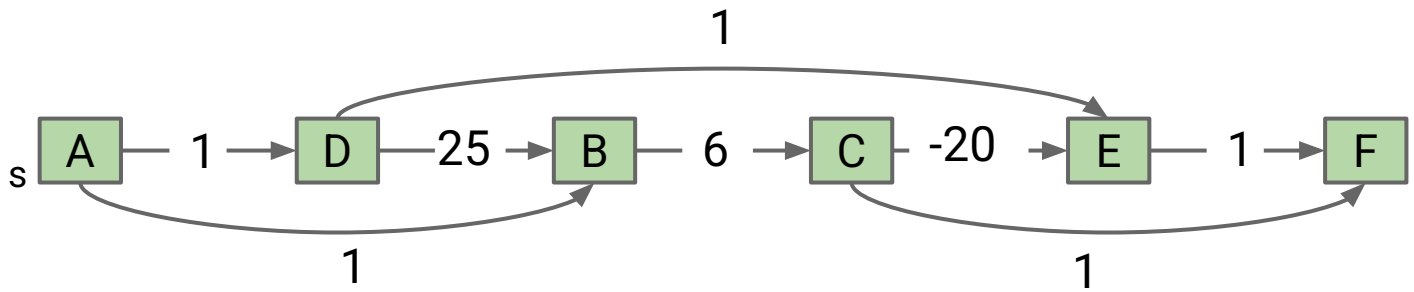
First: We have to find a topological order, e.g. ADBCEF. Runtime is O(V + E).





The DAG SPT Algorithm: Relax in Topological Order

Second: We have to visit all the vertices in topological order, relaxing all edges as we go. Let's see a demo.

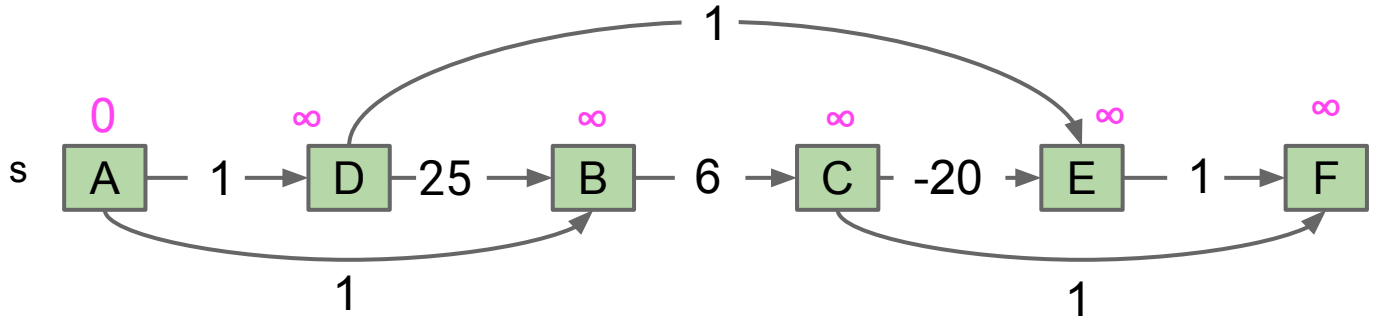


DAG SPT Algorithm

Visit vertices in topological order.

- When we visit a vertex: relax all of its going edges.

	distTo	edgeTo
A	0	-
B	∞	-
C	∞	-
D	∞	-
E	∞	-
F	∞	-



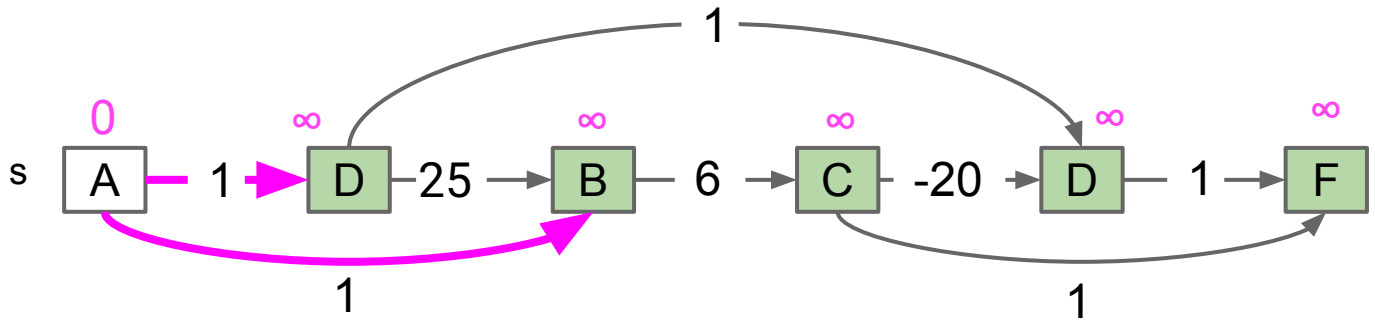
Fringe: [A, D, B, C, E, F]

DAG SPT Algorithm

Visit vertices in topological order.

- When we visit a vertex: relax all of its going edges.

	distTo	edgeTo
A	0	-
B	∞	-
C	∞	-
D	∞	-
E	∞	-
F	∞	-



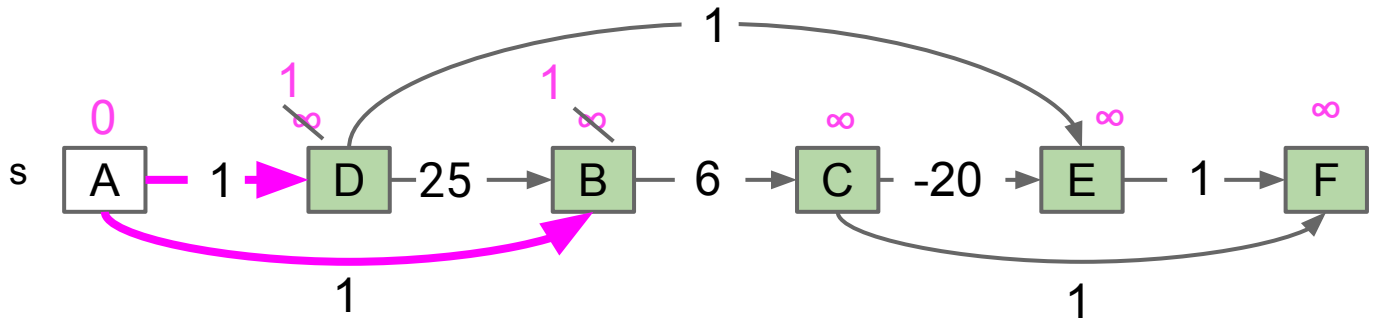
Fringe: [~~A~~, D, B, C, E, F]

DAG SPT Algorithm

Visit vertices in topological order.

- When we visit a vertex: relax all of its going edges.

	distTo	edgeTo
A	0	-
B	1	A
C	∞	-
D	1	A
E	∞	-
F	∞	-



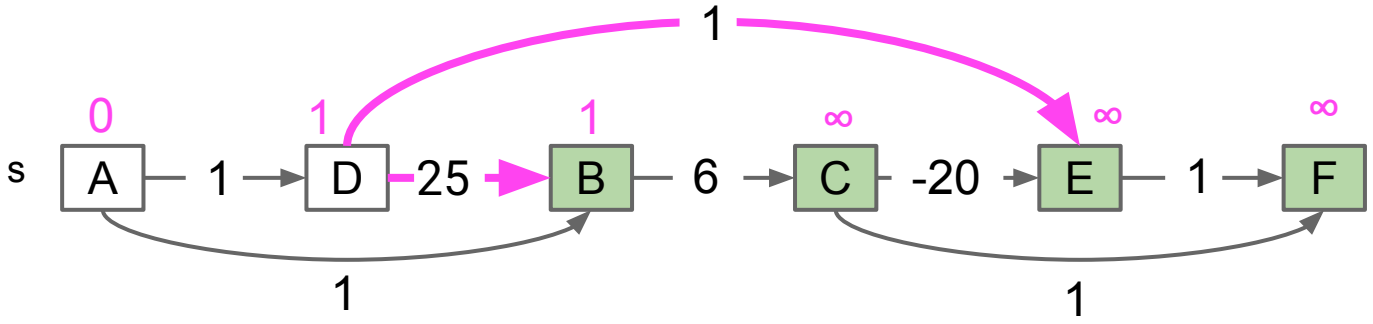
Fringe: [~~A~~, D, B, C, E, F]

DAG SPT Algorithm

Visit vertices in topological order.

- When we visit a vertex: relax all of its going edges.

	distTo	edgeTo
A	0	-
B	1	A
C	∞	-
D	1	A
E	∞	-
F	∞	-



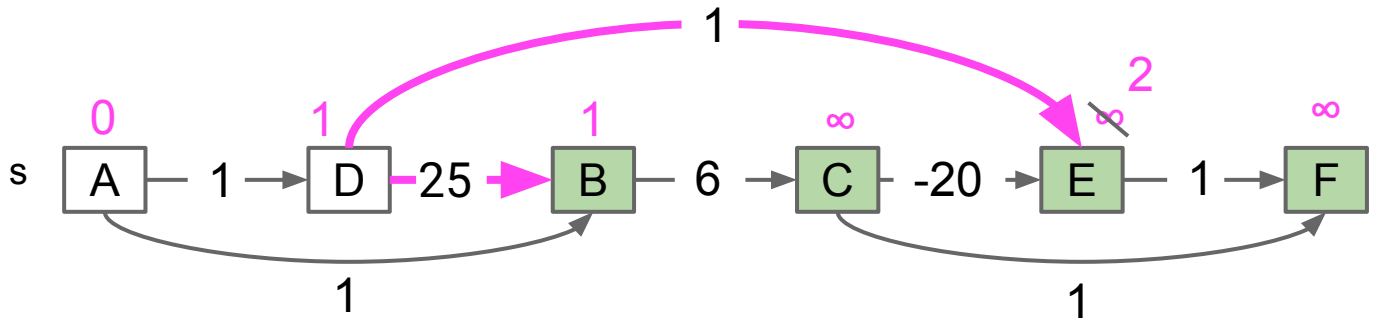
Fringe: [~~A~~, ~~D~~, B, C, E, F]

DAG SPT Algorithm

Visit vertices in topological order.

- When we visit a vertex: relax all of its going edges.

	distTo	edgeTo
A	0	-
B	1	A
C	∞	-
D	1	A
E	2	D
F	∞	-



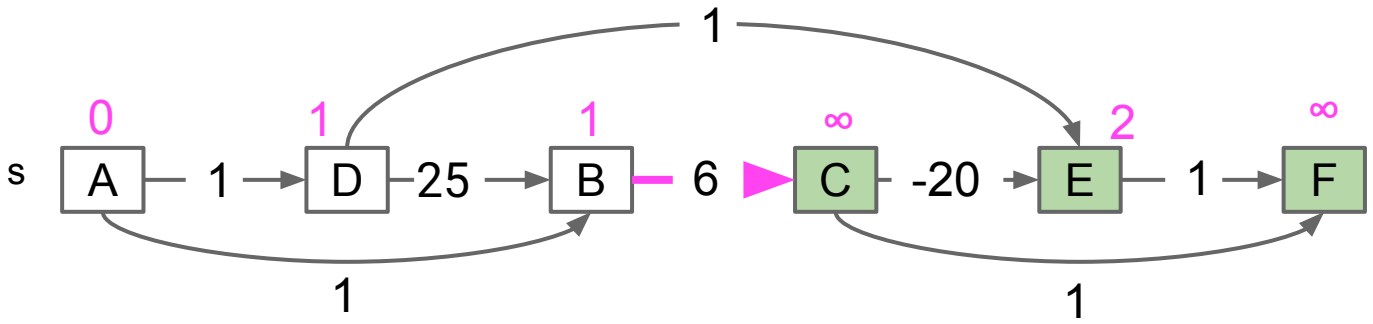
Fringe: [~~A~~, ~~D~~, B, C, E, F]

DAG SPT Algorithm

Visit vertices in topological order.

- When we visit a vertex: relax all of its going edges.

	distTo	edgeTo
A	0	-
B	1	A
C	∞	-
D	1	A
E	2	D
F	∞	-



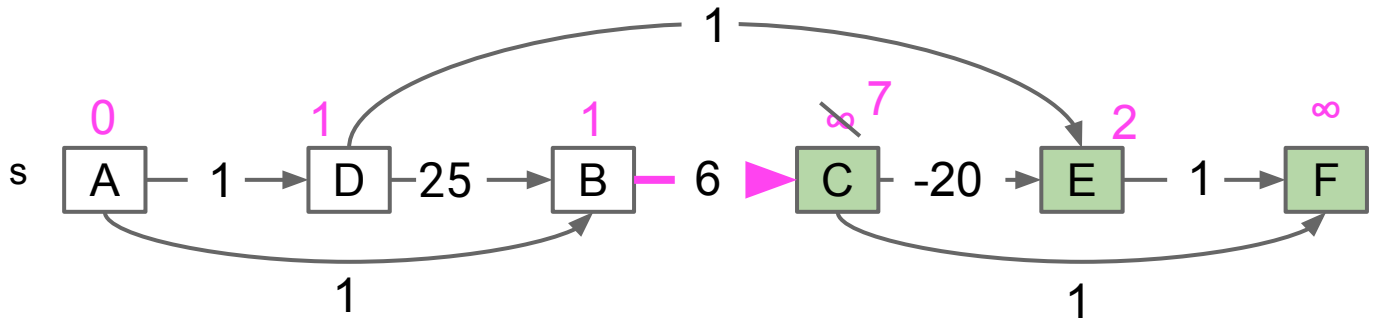
Fringe: [~~A~~, ~~D~~, ~~B~~, C, E, F]

DAG SPT Algorithm

Visit vertices in topological order.

- When we visit a vertex: relax all of its going edges.

	distTo	edgeTo
A	0	-
B	1	A
C	7	B
D	1	A
E	2	D
F	∞	-



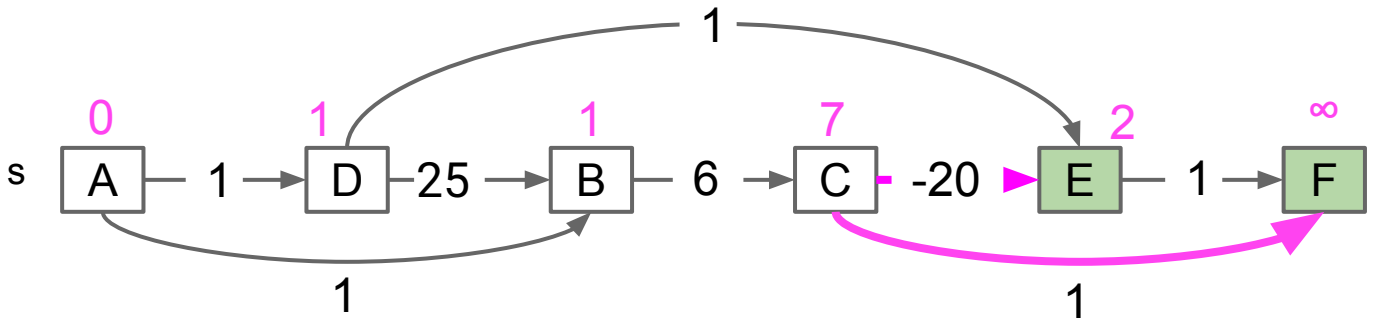
Fringe: [~~A~~, ~~D~~, ~~B~~, C, E, F]

DAG SPT Algorithm

Visit vertices in topological order.

- When we visit a vertex: relax all of its going edges.

	distTo	edgeTo
A	0	-
B	1	A
C	7	B
D	1	A
E	2	D
F	∞	-



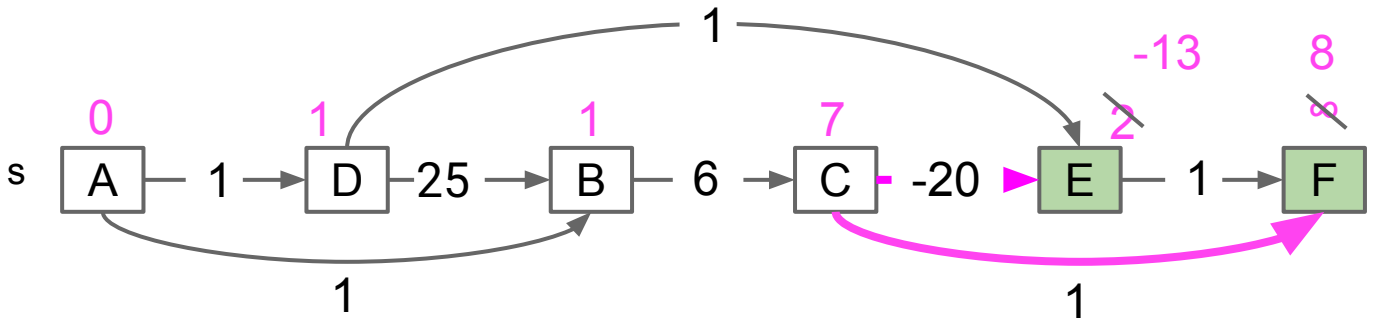
Fringe: [~~A~~, ~~D~~, ~~B~~, ~~C~~, E, F]

DAG SPT Algorithm

Visit vertices in topological order.

- When we visit a vertex: relax all of its going edges.

	distTo	edgeTo
A	0	-
B	1	A
C	7	B
D	1	A
E	-13	C
F	∞	-



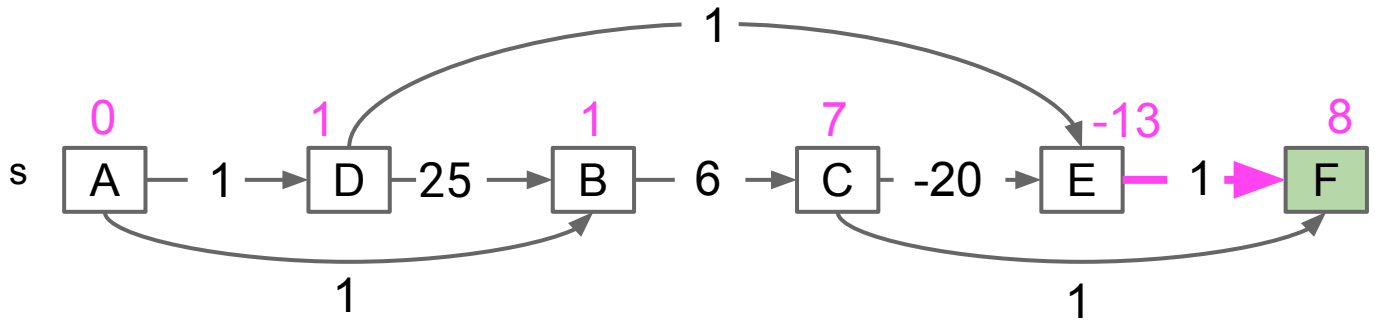
Fringe: [~~A~~, ~~D~~, ~~B~~, ~~C~~, E, F]

DAG SPT Algorithm

Visit vertices in topological order.

- When we visit a vertex: relax all of its going edges.

	distTo	edgeTo
A	0	-
B	1	A
C	7	B
D	1	A
E	-13	C
F	∞	-



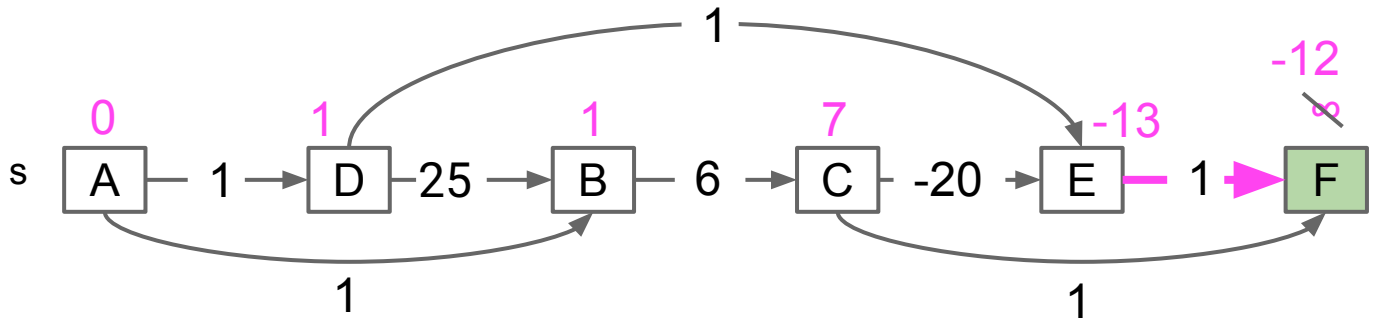
Fringe: [~~A~~, ~~D~~, ~~B~~, ~~C~~, ~~E~~, F]

DAG SPT Algorithm

Visit vertices in topological order.

- When we visit a vertex: relax all of its going edges.

	distTo	edgeTo
A	0	-
B	1	A
C	7	B
D	1	A
E	-13	C
F	-12	E



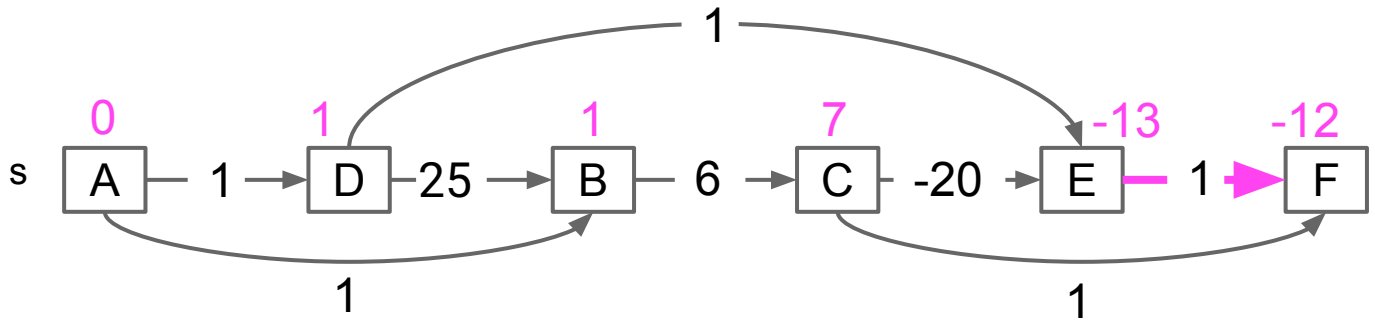
Fringe: [~~A~~, ~~D~~, ~~B~~, ~~C~~, ~~E~~, F]

DAG SPT Algorithm

Visit vertices in topological order.

- When we visit a vertex: relax all of its going edges.

	distTo	edgeTo
A	0	-
B	1	A
C	7	B
D	1	A
E	-13	C
F	-12	E



Fringe: [~~A~~, ~~D~~, ~~B~~, ~~C~~, ~~E~~, ~~F~~]

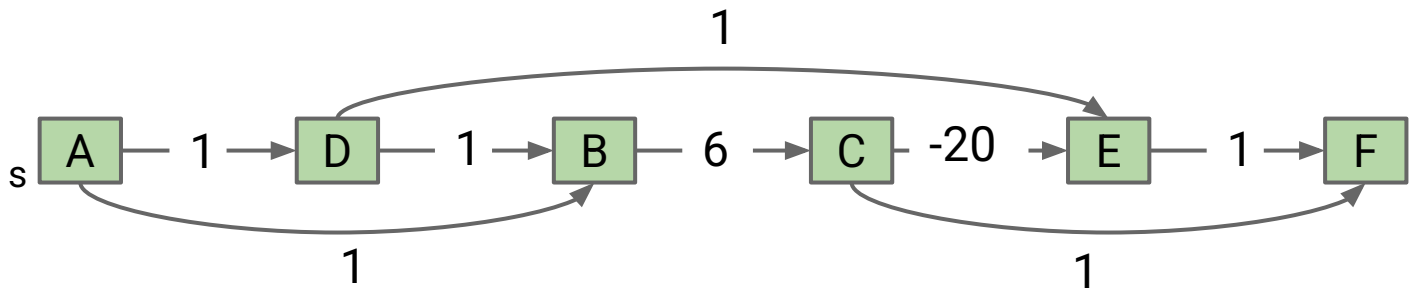
The DAG SPT Algorithm: Relax in Topological Order

Second: We have to visit all the vertices in topological order, relaxing all edges as we go.

- Runtime for step 2 is also O(V + E).

Occasional question asked by students: why isn't it O(V*E)? We're relaxing all edges from each vertex.

- Keep in mind that E is the *total* number of edges in the entire graph, not the number of edges per vertex. Example: for the graph below, E = 8.



Graph Problems

Problem	Problem Description	Solution	Efficiency
topological sort	Find an ordering of vertices that respects edges of our DAG.	Demo Code: Topological.java	O(V+E) time Θ(V) space
DAG shortest paths	Find a shortest paths tree on a DAG.	Demo Code: AcyclicSP.java	O(V+E) time Θ(V) space

Note: The DAG shortest paths solution uses the topological sort solution as a subroutine.

What If It's Not a DAG?

The DAG algorithm only works on graphs that contain no cycles.

- If a graph has negative edges, and includes some cycles, can't use DAG algorithm.
- Finding a fast algorithm on such graphs was an open problem for 40 years, solved in 2022 by Bernstein, Nanongkai and Wulff-Nilsen.
  - Read more here: https://www.quantamagazine.org/finally-a-fast-algorithm-for-shortest-paths-on-negative-graphs-20230118/

Longest Paths

Lecture 25, CS61B, Spring 2026

Topological Sorting

Shortest Paths on DAGs

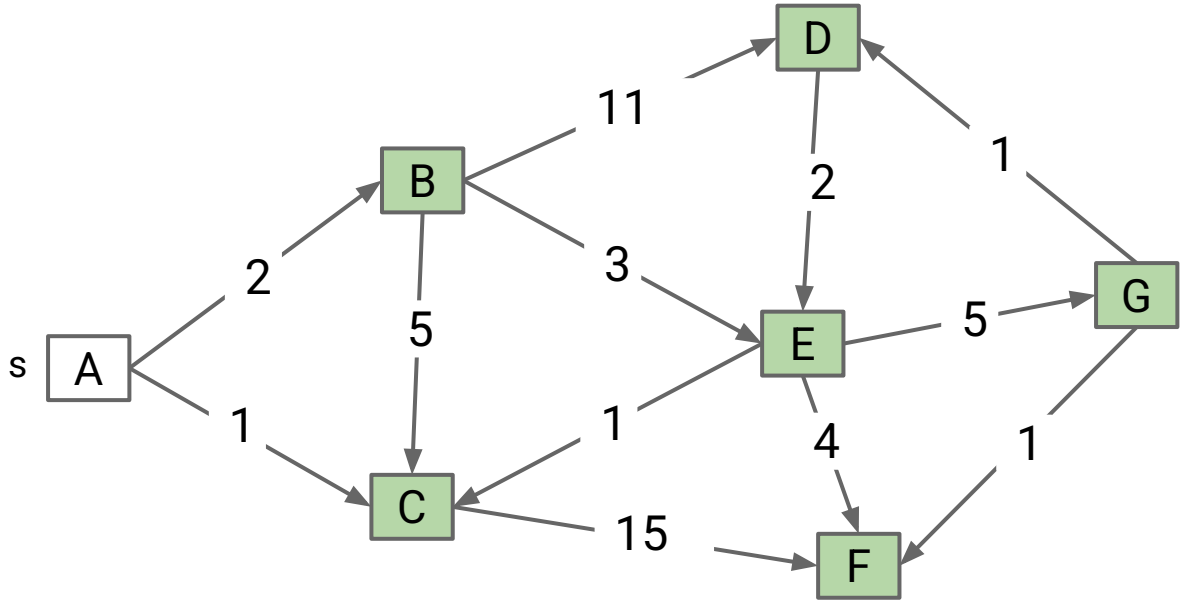
Longest Paths

Reductions

- Definition
- Reduction to 3SAT (Optional CS170 Preview)

The Longest Paths Problem

Consider the problem of finding the longest path tree (LPT) from s to every other vertex. The path must be simple (no cycles!).

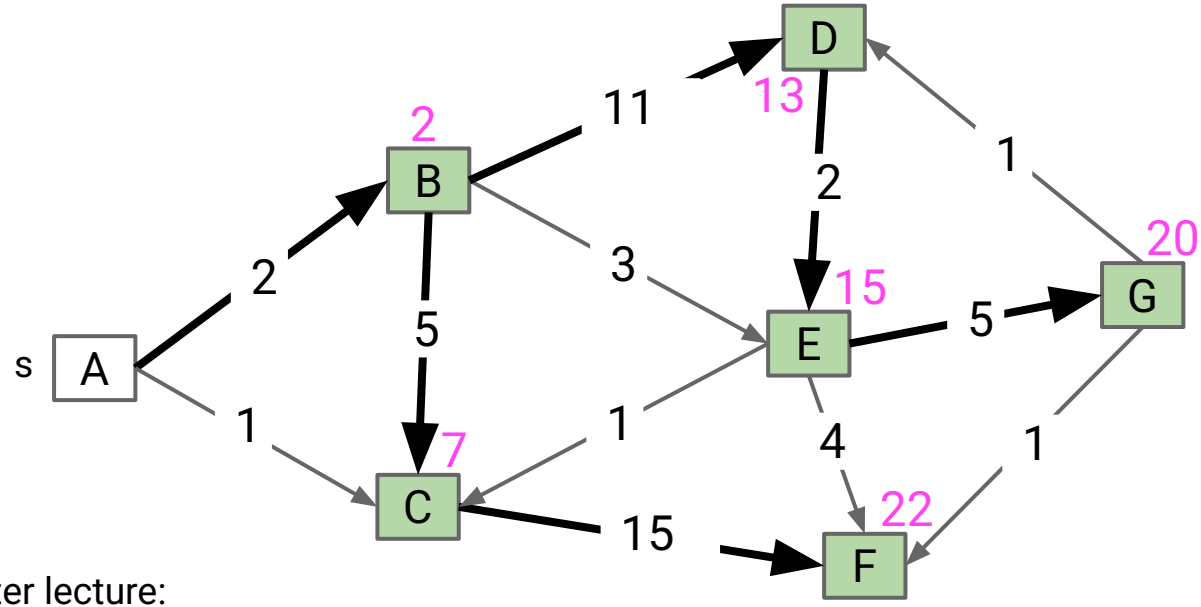


The Longest Paths Problem

Consider the problem of finding the longest path tree (LPT) from s to every other vertex. The path must be simple (no cycles!).

Some surprising facts:

- Best known algorithm is exponential (extremely bad).
- Perhaps the most important unsolved problem in mathematics.



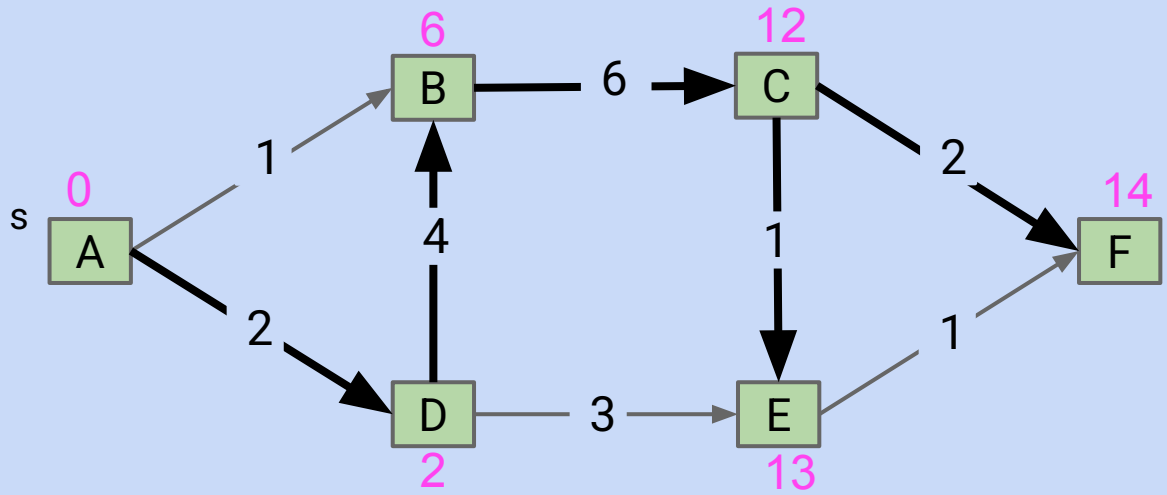
Two potentially interesting exercises after lecture:

- Find an example where the obvious algorithm (Dijkstra's but pick the biggest edge first) fails.
- Figure out: Is the longest path to every other vertex always a tree (i.e. does an LPT exist for all graphs)?

The Longest Paths Problem on DAGs

Difficult challenge for you.

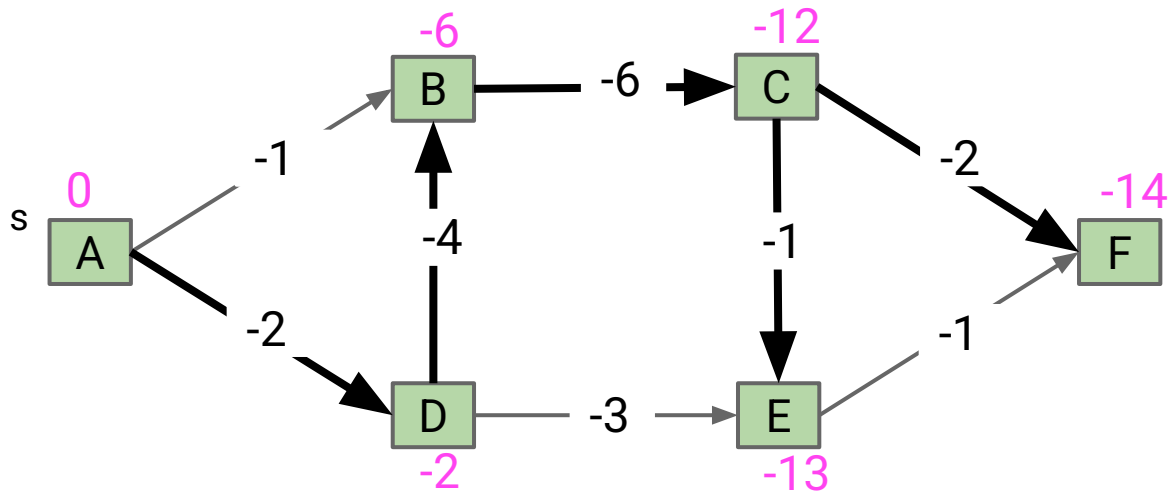
- Solve the LPT problem on a directed acyclic graph.
- Algorithm must be O(E + V) runtime.



The Longest Paths Problem on DAGs

DAG LPT solution for graph G:

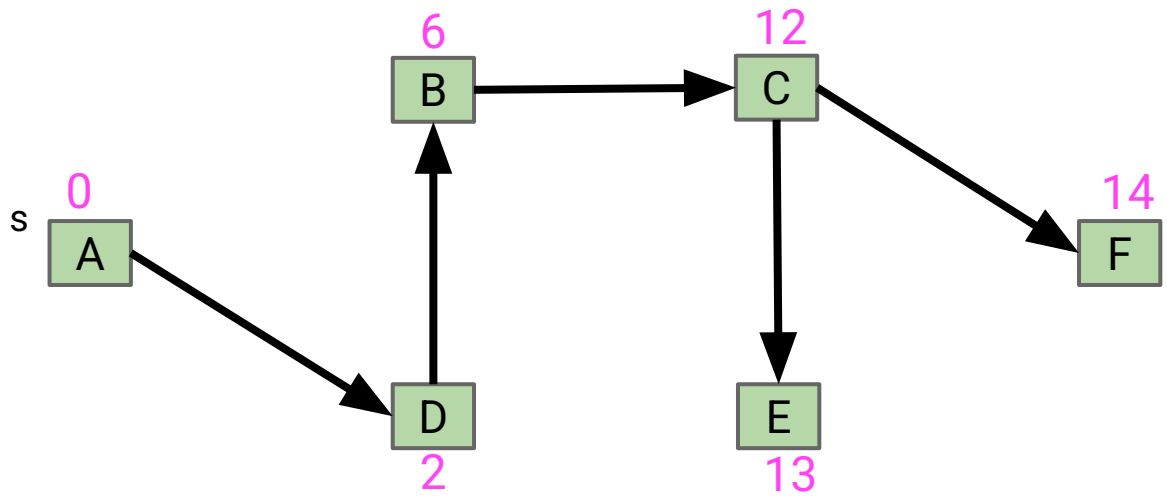
- Form a new copy of the graph G' with signs of all edge weights flipped.
- Run DAGSPT on G' yielding result X.
- Flip signs of all values in X.distTo. X.edgeTo is already correct.



The Longest Paths Problem on DAGs

DAG LPT solution for graph G:

- Form a new copy of the graph G’ with signs of all edge weights flipped.
- Run DAGSPT on G’ yielding result X.
- Flip signs of all values in X.distTo. X.edgeTo is already correct.



A Note on “Mathematical Maturity”

If you have a very high degree of so-called “mathematical maturity”, this algorithm should seem plainly correct.

There’s no real need to prove anything or show demos.

- We know DAG SPT works on graphs with negative edge weights.
- We also know that -(-a + -b + -c + -d) = a + b + c + d.

Part of what you’re learning in your intense technical education here at Berkeley is mathematical maturity. Hasn’t been a major focus in 61B, but will be in other courses like EECS 16A, EECS 16B, CS 126, CS 127, CS 70, CS 170, Math 110, ...

Highly Recommended Exercise For Later

Play around with the longest paths problem and convince yourself that it is actually very hard.

- Try to develop an intuition for why it is hard. Even better if you try to put it into english.
- Try searching the internet for “why longest paths hard” or similar if you’re having trouble really pinning down what’s so hard about it.

Graph Problems

Problem	Problem Description	Solution	Efficiency
topological sort	Find an ordering of vertices that respects edges of our DAG.	Demo Code: Topological.java	O(V+E) time Θ(V) space
DAG shortest paths	Find a shortest paths tree on a DAG.	Demo Code: AcyclicSP.java	O(V+E) time Θ(V) space
longest paths	Find a longest paths tree on a graph.	No known efficient solution.	O(???) time O(???) space
DAG longest paths	Find a longest paths tree on a DAG.	Flip signs, run DAG SPT, flip signs again.	O(V+E) time Θ(V) space

Reductions: Definition

Lecture 25, CS61B, Spring 2026

Topological Sorting

Shortest Paths on DAGs

Longest Paths

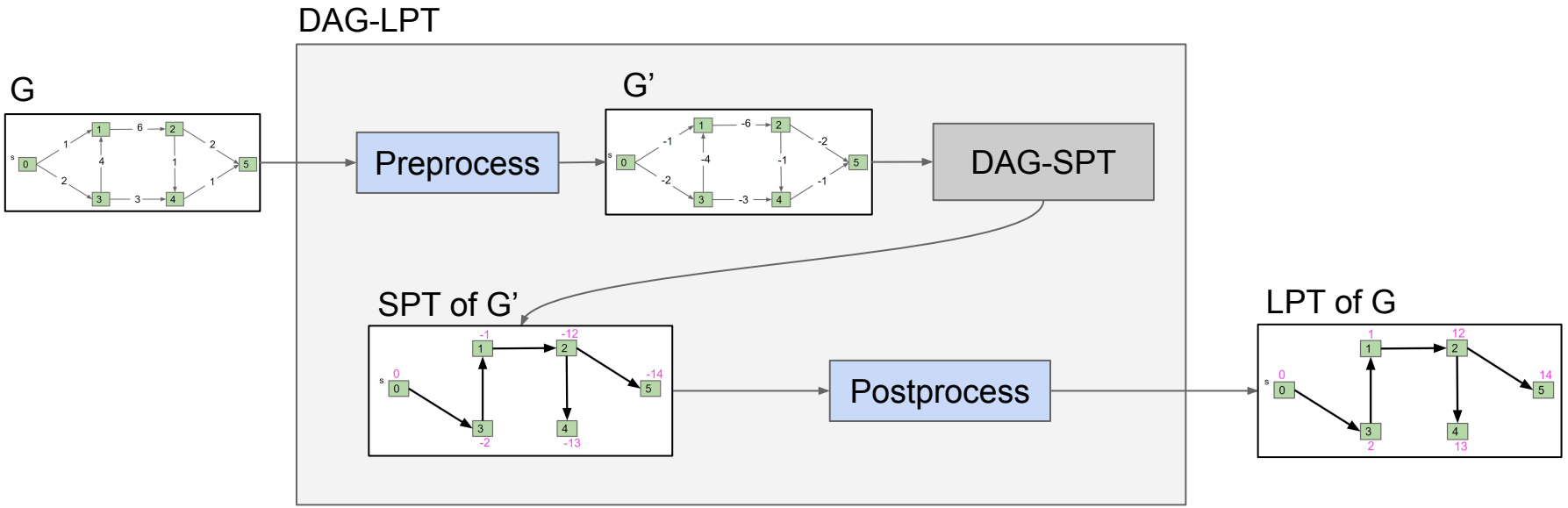
Reductions

- **Definition**
- Reduction to 3SAT (Optional CS170 Preview)

DAG Longest Paths

The problem solving we just used probably felt a little different than usual:

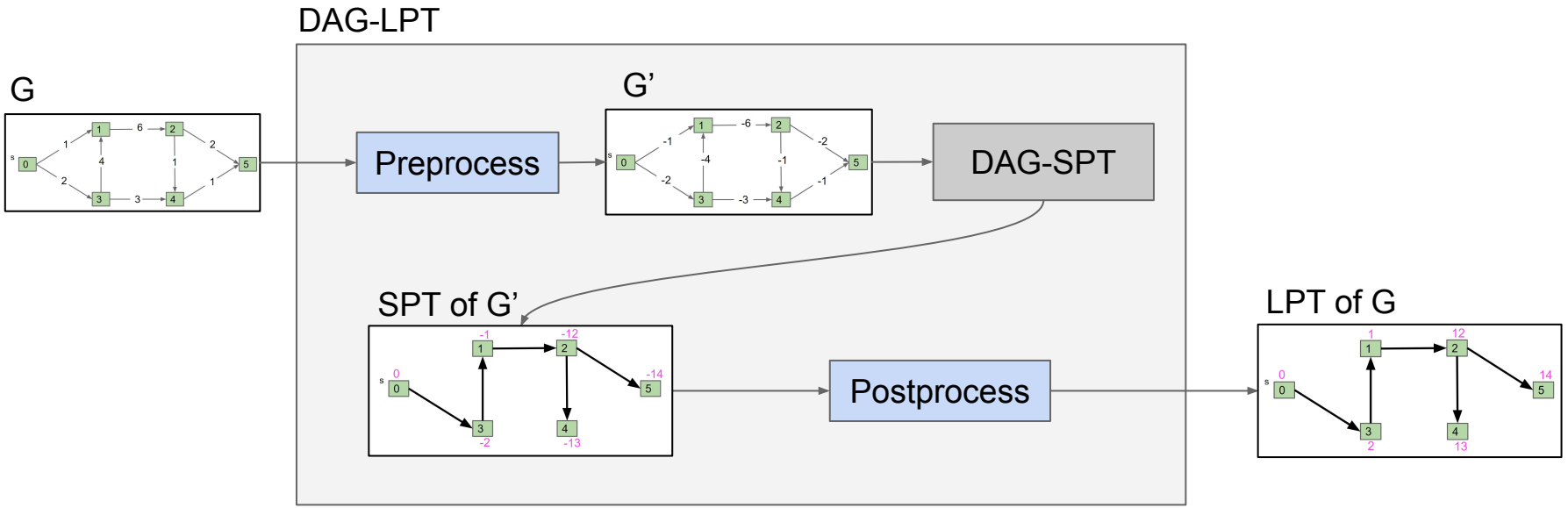
- Given a graph G, we created a new graph G’ and fed it to a related (but different) algorithm, and then interpreted the result.



Reduction

This process is known as **reduction**.

- Since DAG-SPT can be used to solve DAG-LPT, we say that “DAG-LPT reduces to DAG-SPT”.



Reduction Analogy

This process is known as reduction.

- Since DAG-SPT can be used to solve DAG-LPT, we say that “DAG-LPT reduces to DAG-SPT”.

As a real-world analog, suppose we want to climb a hill. There are many ways to do this:

- “Climbing a hill” reduces to “riding a ski lift.”
- “Climbing a hill” reduces to “being shot out of a cannon.”
- “Climbing a hill” reduces to “riding a bike up the hill.”

Reduction Definition (Informal)

This process is known as reduction.

- Since DAG-SPT can be used to DAG-LPT, we say that “DAG-LPT reduces to DAG-SPT”.

Algorithms by Dasgupta, Papadimitriou, and Vazirani defines a reduction informally as follows:

- “If any subroutine for task Q can be used to solve P, we say P reduces to Q.”

Can also define the idea formally, but **way** beyond the scope of our class.

- If you’re curious, you can read more about Karp and Cook reductions.

Reduction is more than sign flipping. In the bonus slides, we give an example of a richer reduction (3SAT to Independent Set)

Reductions

Earlier I claimed that the longest paths problem is potentially the most important unsolved question in mathematics.

- This is true, but also a big misleading.
- The reason: Vast numbers of important problems reduce to the longest paths problem!
- But also: The longest path problem also reduces back to those!
- In other words: Solving any of these problems solves them all.

We might revisit this idea (NP-Completeness) in the last week of class.

Reductions and Decomposition

Reductions are just a more formal version of what we've been doing throughout the course.

- Percolation can be solved with Disjoint Sets.
- Finding the hyponyms of a word can be solved with DFS (or BFS).

At its heart, computer science is about taking a complex task and breaking it into smaller parts.

- Using appropriate abstractions makes problem solving vastly easier.

Generational Changes in the Human Mind

Interesting question: Are we regressing? Are LLMs partially to blame?

Reminder

Project 4C due Tuesday after Spring Break!.

Reduction to 3SAT (Optional CS170 Preview)

Lecture 25, CS61B, Spring 2026

Topological Sorting

Shortest Paths on DAGs

Longest Paths

Reductions

- Definition
- **Reduction to 3SAT (Optional CS170 Preview)**

The Independent Set Problem

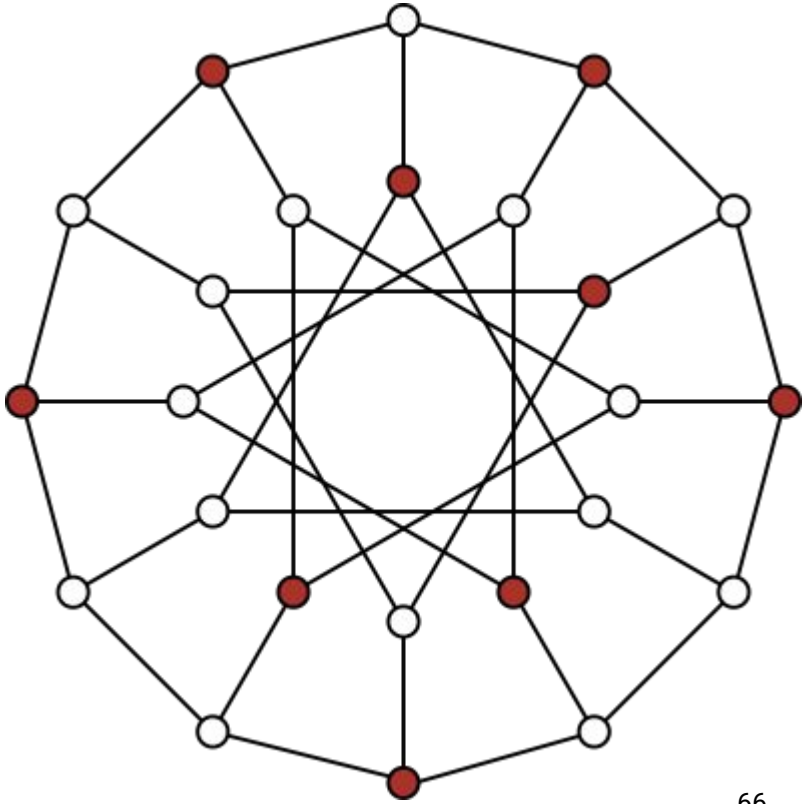
An independent set is a set of vertices in which no two vertices are adjacent.

The Independent-set Problem:

- Does there exist an independent set of size k?
- i.e. color k vertices red, such that none touch.

Example for the graph on the right and k = 9

- For this particular graph, N=24.



The 3SAT Problem

3SAT: Given a boolean formula, does there exist a truth value for boolean variables that obeys a set of 3-variable disjunctive constraints?

3 variable disjunctive constraint

Example: (x1 || x2 || !x3) && (x1 || !x1 || x1) && (x2 || x3 || x4)

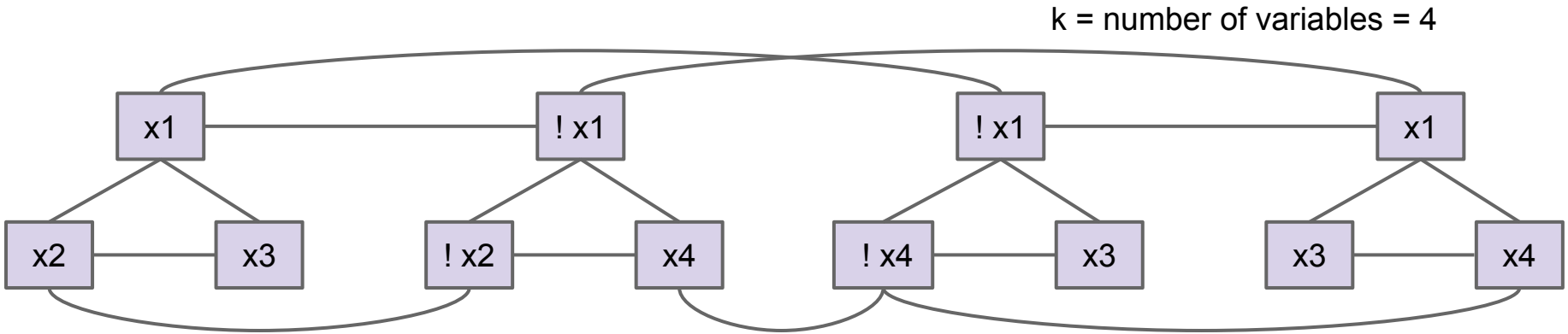
- Solution: x1 = true, x2 = true, x3 = true, x4 = false

3SAT Reduces to Independent Set

Proposition: 3SAT reduces to Independent-set.

Proof. Given an instance ϕ of 3-SAT, create an instance G of Independent-set:

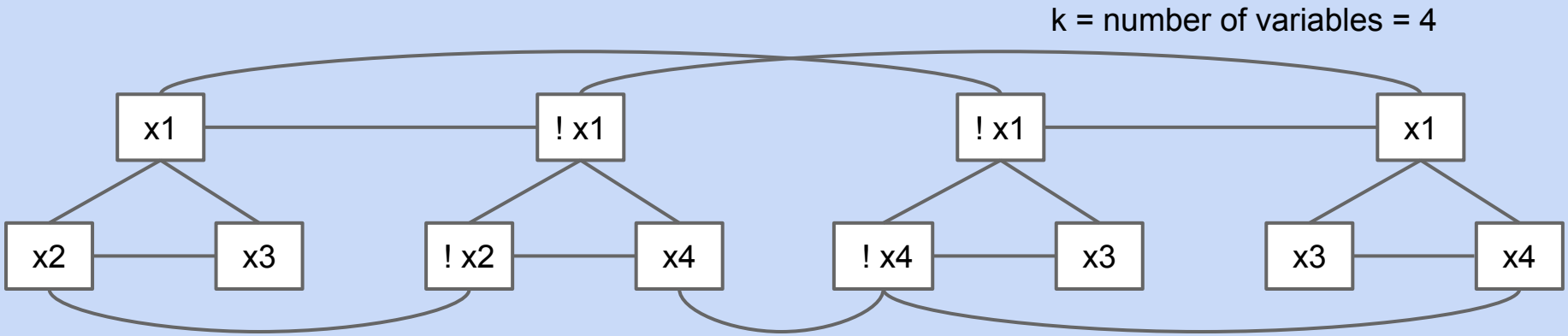
- For each clause in ϕ, create 3 vertices in a triangle.
- Add an edge between each literal and its negation (can't both be true in 3SAT means can't be in same set in Independent-set)



3SAT Reduces to Independent Set

Find an independent set of size k = 4. Use this set to generate a solution to the 3SAT problem.

- Reminder: An independent set of size 4 is a set of 4 (red) vertices that do not touch.

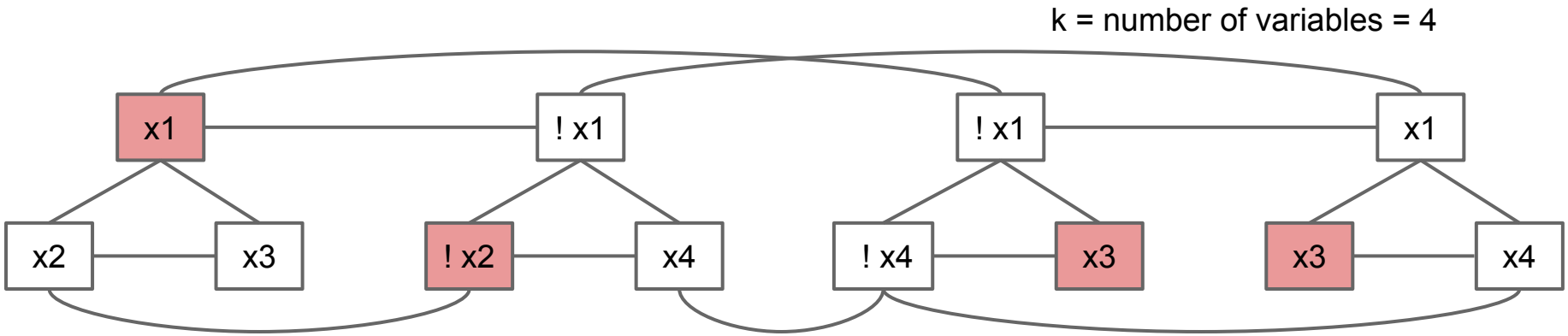


Φ = (x1 or x2 or x3) and (! x1 or ! x2 or x4) and (! x1 or x3 or ! x4) and (x1 or x3 or x4)

3SAT Reduces to Independent Set

Find an independent set of size k = 4. Use this set to generate a solution to the 3SAT problem.

- Reminder: An independent set of size 4 is a set of 4 (red) vertices that do not touch.



Φ = (x1 or x2 or x3) and (! x1 or ! x2 or x4) and (! x1 or x3 or ! x4) and (x1 or x3 or x4)

Reduction

Since IND-SET can be used to solve 3SAT, we say that “3SAT reduces to IND-SET”.

- Note: 3SAT is not a graph problem!
- Note: Reductions don’t always involve creating graphs.